

Week10_Ch05. 자연어 처리-토큰화

5.1. 자연어 처리와 토큰화 개요

5.1.1. 자연어와 자연어 처리

정의

- **자연어[자연 언어, National Language]**: 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- **자연어 처리(Natural Language Processing, NLP)**: 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

5.1.2. 자연어 처리[NLP] 개념

- 인공지능의 하위 분야 중 하나
- 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표 중 하나
- 인간 언어의 구조, 의미, 맥락을 분석하고 이해할 수 있는 알고리즘과 모델을 개발
 - ⇒ 모호성, 가변성, 구조에 대한 문제 해결이 필요

해결해야 하는 문제

1. 모호성(Ambiguity)

- 인간의 언어는 단어와 구가 사용되는 맥락에 따라 여러 의미를 갖게 되어 모호한 경우가 많음
- 알고리즘은 이러한 다양한 의미를 이해하고 명확하게 구분할 수 있어야 함

2. 가변성(Variability)

- 인간의 언어는 다양한 사투리(Dialects), 강세(Accent), 신조어(Coined word), 작문 스타일로 인해 매우 가변적임
- 알고리즘은 이러한 가변성을 처리할 수 있어야 하며, 사용 중인 언어를 이해할 수 있어야 함

3. 구조(Structure)

- 인간의 언어는 문장이나 구의 의미를 이해할 때 구문(Syntactic)을 파악하며, 의미(Semantic)를 해석함
- 알고리즘은 문장의 구조와 문법적 요소를 이해하여 의미를 추론하거나 분석할 수 있어야 함

⇒ 이와 같은 문제를 이해하고 구분할 수 있는 모델을 만들려면 **말뭉치(Corpus)**를 일정한 단위인 **토큰(Token)**으로 나뉘야 함

5.1.3. 토큰화

말뭉치(Corpus)

: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어

- 뉴스 기사, 사용자 리뷰, 저널, 칼럼 등에서 목적에 따라 구축되는 텍스트 데이터
- 일련의 단어의 가능성을 예측하는 알고리즘인 언어 모델을 구축, 평가하는 데 자주 사용

토큰(Token)

: 개별 단어, 문장 부호와 같은 텍스트

- 말뭉치보다 더 작은 단위
- 텍스트의 개별 단어, 구두점, 기타 의미 단위일 수 있음

토큰화(Tokenization)

: 토큰으로 나누는 것

- 컴퓨터가 자연어를 이해할 수 있게 나누는 과정
- 컴퓨터가 텍스트를 효율적으로 분석하고 처리할 수 있도록 하는 중요한 단계
- 자연어 처리 과정에서 중요한 단계

5.1.4. 토큰나이저(Tokenizer)

: 텍스트 문자열을 토큰으로 나누는 알고리즘/소프트웨어

- 토큰화를 위해 사용
- 텍스트에서 발생하는 다양한 언어와 구문을 식별하고 분석할 수 있게 함
⇒ 언어 모델링, 기계 번역과 같은 다양한 자연어 처리 작업에서 사용
- 예)

입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'

결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '!']

토큰화 구축 방법

- 토큰을 나누는 기준을 구축하려는 시스템, 상황에 따라 다름
- 어떻게 토큰을 나누었느냐에 따라 시스템의 성능이나 처리 결과가 크게 달라지기도 함

1. 공백 분할

: 텍스트를 공백 단위로 분리해 개별 단어로 토큰화

2. 정규표현식 적용

: 정규 표현식으로 특정 패턴을 식별해 텍스트를 분할

3. 어휘 사전(Vocabulary) 적용

: 사전에 정의된 단어 집합을 토큰으로 사용

- 직접 어휘 사전을 구축하기 때문에 없는 단어, 토큰이 존재할 수 있음
⇒ 없는 토큰은 OOV(Out of Vocab)이라고 함
⇒ OOV 문제를 고려하여 토큰화해야 함
- 큰 어휘 사전을 구축하면 학습 비용의 증대, 차원의 저주(Curse of Dimensionality)에 빠질 수 있음
 - 토큰, 단어를 벡터화하면 어휘 사전에 등장하는 토큰 개수만큼의 차원이 필요
 - 벡터값이 거의 모두 0의 값을 가지는 희소(sparse) 데이터로 표현
 - 희소 데이터의 표현 방법은 어휘 사전에 등장하는 출현 빈도만 고려
⇒ 문장에서 발생한 토큰들의 순서 관계를 잘 표현하지 못함

4. 머신러닝 활용

: 데이터셋을 기반으로 토큰화하는 방법을 학습한 머신러닝을 적용

5.2. 단어 및 글자 토큰화

- 입력된 텍스트 데이터를 단어(Word)나 글자(Character) 단위로 나누는 기법
 - 단어 토큰화
 - 글자 토큰화
⇒ 각각의 토큰을 의미를 갖는 최소 단위로 분해
- 더 정교한 토큰화 기법이 연구되고 있지만, 여전히 단어 토큰화나 글자 토큰화처럼 간단하고 명료한 아이디어가 강력한 토큰화 기법으로 사용되고 있음

5.2.1. 단어 토큰화(Word Tokenization)

개념

- 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나
- 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업
- 대부분의 언어는 띄어쓰기를 이용해 문장을 의미 있는 단어로 나눠 표현

⇒ 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행

- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용
- 가장 일반적인 토큰화 방법

토큰화 방법

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = review.split( )
print(tokenized)

'''
출력 결과
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
'''
```

- **split 메서드**를 이용해 쉽게 토큰화 가능
 - 주어진 구분자를 통해 문자열을 리스트 데이터로 나눔
 - 구분자를 입력하지 않으면 공백을 기준으로 나눔
- '최고!'와 '최고!!'는 느낌표 하나 차이로 다른 의미 단위로 나뉨
- 'cg', 'cg.'도 다른 단어 토큰으로 나뉨
 - ⇒ 단어 토큰화를 통해 만들어진 단어 사전에서 두 토큰은 다른 토큰
 - ⇒ 'cg'라는 토큰이 단어 사전 내에 있더라도 'cg.', 'cg는' 등은 OOV가 됨
 - ⇒ 단어 토큰화는 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류 등에 취약함

5.2.2. 글자 토큰화(Character Tokenization)

개념

- 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식
- 비교적 작은 단어 사전을 구축할 수 있음
 - 컴퓨터 자원을 아낄 수 있음
 - 전체 말뭉치를 학습할 때 각 단어를 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용
 - 다음 문자를 예측하는 언어 모델링에서 글자 토큰화는 유용한 방식

토큰화 방법

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
tokenized = list(review)
print(tokenized)

'''
출력 결과
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불', '어', ' ', 'o', 's', 't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!', '!']
'''
```

- 리스트 형태로 변환하면 쉽게 토큰화 가능
- 단어 토큰화와는 다르게 공백도 토큰으로 나뉨
- 영어는 글자 토큰화를 진행하면 각 알파벳으로 나뉘지만, 한글은 하나의 글자가 여러 자음과 모음의 조합으로 이루어져 있어 자소 단위로 나뉘 **자소 단위 토큰화**를 수행
 - ⇒ **자모(jamo) 라이브러리** 활용

- 한글 문자, 자모 작업을 위한 한글 음절 분해 및 합성 라이브러리

자모(jamo) 라이브러리 활용

- 자모 변환 함수(jamo.h2j)

```
retval = jamo.h2j(
    hangul_string
)
```

- 입력된 한글 문자열을 유니코드 U+100 ~ U+11FE 사이의 조합형 한글 자모로 변환하는 함수

<컴퓨터가 한글을 인코딩하는 방식>

- 조합형
 - : 글자를 자모 단위로 나눠 인코딩한 뒤 이를 조합해 한글을 표현
- 완성형
 - : 조합된 글자 자체에 값을 부여해 인코딩

- h2j 함수는 완성형으로 입력된 한글을 조합형 한글로 변환

- 한글 호환성 자모 변환 함수(jamo.j2hcj)

```
retval = jamo.j2hcj(
    jamo
)
```

- 조합형 한글 문자열을 자소 단위로 나눠 반환하는 함수
- 조합형 한글로 입력된 문자열은 초성, 중성, 종성으로 나뉨
- 이 함수를 통해 완성형 한글을 쉽게 자소 단위로 나눌 수 있음

- 자소 단위 토큰화 예제

```
from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)
```

- 출력 결과

```
['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄴ', ' ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㄱ', 'ㅏ', 'ㅇ', 'ㅎ', 'ㅏ', 'ㄴ', ' ', 'ㄷ', 'ㄱ', 'ㅇ', 'ㅅ', 'ㅣ', 'ㄱ', 'ㅏ', 'ㄱ', 'ㅈ', 'ㅌ', 'ㄱ', ' ', 'ㅈ', 'ㅡ', 'ㄹ', 'ㄱ', 'ㅌ', 'ㅇ', 'ㅡ', 'ㅁ', 'ㅇ', 'ㅡ', 'ㄴ', ' ', 'ㅈ', 'ㅌ', 'ㅌ', 'ㄱ', 'ㅏ', '!', ' ', 'ㄷ', 'ㅌ', 'ㅂ', 'ㅌ', 'ㄹ', 'ㅇ', 'ㅌ', ' ', 'ㅇ', 'ㅅ', 'ㅌ', 'ㄴ', 'ㅡ', 'ㄴ', ' ', 'ㄷ', 'ㅌ', 'ㄷ', 'ㅌ', 'ㅇ', 'ㅌ', 'ㄱ', ' ', 'ㅈ', 'ㅌ', 'ㄱ', 'ㅏ', '!', '!']
```

글자 토큰화의 장점

- 단어 토큰화에 비해 적은 크기의 단어 사전을 구축할 수 있음
- 접사와 문장 부호의 의미를 학습할 수 있음
 - 'cg도', 'cg는', 'cg.' 에서의 '도', '는', '.'
- 작은 크기의 단어 사전으로도 OOV를 획기적으로 줄일 수 있음

글자 토큰화의 단점

- 개별 토큰은 아무런 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 함
 - 토큰 조합 방식을 사용해 문장 생성이나 개체명 인식(Name Entity Recognition) 등을 구현할 경우, 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려울 수 있음
- 모델 입력 시퀀스의 길이가 길어질수록 연산량이 증가
 - 예시) `review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"`
 - 단어 토큰화의 입력 시퀀스 길이: 13
 - 글자 토큰화의 입력 시퀀스 길이: 53
 - 자소 단위 토큰화의 입력 시퀀스 길이: 101

5.3. 형태소 토큰화(Morpheme Tokenization)

5.3.1. 형태소와 형태소 토큰화

형태소 토큰화

- 텍스트를 형태소 단위로 나누는 토큰화 방법
 - 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류하는 작업
 - 한국어와 같이 교착어(Agglutinative Language)인 언어에서 중요하게 수행됨
 - 한국어는 대부분의 언어와 달리 각 단어가 띄어쓰기로 구분되지 않음
 - 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이룸
- ⇒ 각 형태소를 적절히 구분해 처리해야 함

형태소

- 한국어는 어근+접사/조사로 조합되어 하나의 낱말을 이룸
 - 예) '그는 나에게 인사를 했다'
 - '그는' = '그'(단어) + '는'(조사)
 - '나에게' = '나'(단어) + '에게'(조사)
- ⇒ '그', '-는', '나', '-에게' 등과 같이 실제로 의미를 가지고 있는 최소의 단위를 **형태소(Morpheme)**라고 함
- **형태소 분류**
 - **자립 형태소(Free Morpheme)**
: 스스로 의미를 가지는 형태소
 - 단어의 기본이 되는 형태소
 - 명사, 동사, 형용사와 같은 단어를 이루는 기본 단위
 - **의존 형태소(Bound Morpheme)**
: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 형태소
 - 자립 형태소와 함께 조합되어 문장에서 특정한 역할을 수행
 - 조사, 어미, 접두사, 접미사 등이 해당
 - 형태소 분석을 통해 문장 내 각 형태소의 역할을 파악할 수 있고, 이를 바탕으로 문장을 이해하고 처리할 수 있음

5.3.2. 형태소 어휘 사전(Morpheme Vocabulary)

개념

- 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중에서도 각 단어의 형태소 정보를 포함하는 사전
- 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있음
- 형태소 분석 작업에서 매우 중요한 역할을 함

다른 어휘 사전과의 차이점

"그는 나에게 인사를 했다"
"나는 그에게 인사를 했다"

- **띄어쓰기를 활용해 토큰화 수행(단어 토큰화)**

⇒ 어휘 사전: ['그는', '나에게', '인사를', '했다', '나는', '그에게']

- "그녀는 나에게 인사를 했다" 문장 추가

⇒ '그는', '나는', '그녀는'과 같은 데이터를 같은 의미 단위로 인식하지 못하고 학습을 진행

- **형태소 단위로 토큰화 수행**

⇒ 어휘 사전: ['그', '-는', '나', '-에게', '인사', '-를', '했-', '-다']

- "그녀는 나에게 인사를 했다" 문장 추가

⇒ ['-는', '-에게', '그', '나']와 같은 정보에서 '그녀'의 토큰 정보만 새롭게 학습해 '그녀는', '그녀에게'와 같은 어휘를 쉽게 학습할 수 있음

품사 태깅[POS Tagging]

: 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사(Part of Speech, POS)를 태깅하는 작업

- 일반적으로 형태소 어휘 사전에는 각 형태소가 어떤 품사와 속하는지, 해당 품사의 뜻 등의 정보도 함께 제공됨
- 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있어 더욱 정확한 분석이 가능
- 예시)

"그는 나에게 인사를 했다"

=> 그(명사) + 는(조사) + 나(명사) + 에게(조사) + 인사(명사) + 를(조사) + 했다(동사)

5.3.3. KoNLPy 라이브러리 실습

KoNLPy 라이브러리

- 한국어 자연어 처리를 위해 개발된 라이브러리
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능을 제공
- 텍스트 데이터를 전처리하고 분석하기 위한 다양한 도구와 함수를 제공
- 텍스트 마이닝, 감성 분석, 토픽 모델링 등 다양한 NLP 작업에서 사용
- 자바 언어 기반의 한국어 형태소 분석기로 자바 개발 키트(JDK) 기반으로 개발
 - ⇒ 자바 기반의 형태소 분석을 수행
 - ⇒ 자바 개발 키트가 설치되어 있어야 함
- Okt(Open Korean Text), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab) 등 다양한 형태소 분석기 제공

실습

- SNS 텍스트 데이터를 기반으로 개발된 Okt, 국립국어원에서 배포한 세종 말뭉치를 기반으로 학습된 꼬꼬마 사용
- 'Week10_예습과제_방민지.ipynb' 참고

Okt(Open Korean Text)

- 문장을 입력받아 명사, 구, 형태소, 품사 등의 정보를 추출하는 여러 가지 메서드를 제공
 - `okt.nouns`: 명사 추출
 - `okt.phrases`: 구문 추출
 - `okt.morphs`: 형태소 추출
 - `okt.pos`: 품사 태깅

=> 입력된 문장에서 각각 명사, 어절 구 단위, 형태소만 추출해 리스트를 반환

꼬꼬마(Kkma)

- 명사, 문장, 형태소, 품사 추출
- 구문 추출 기능은 지원하지 않음
- 명사 추출(`kkma.sentences`) 기능 제공

Okt v.s. 꼬꼬마

- 명사 추출: Okt에서 반환하던 '널'과 '수'를 반환하지 않았음
 - 형태소 추출: 꼬꼬마는 '할'을 더 분리해 '하', 'ㄹ'로 반환
 - 품사 태깅: Okt는 19개의 품사를 구분해 반환, 꼬꼬마는 56개의 품사로 태깅
- => 태깅하는 품사의 수가 많으면 더 자세한 단위로 분석이 가능, 품사 태깅에 소요되는 시간 길어짐, 더 많은 품사로 분리해 모델의 성능이 저하될 수 있음

표 5.1 Okt 품사 태그

태그	품사	태그	품사
Noun	명사	Punctuation	구두점
Verb	동사	Foreign	외국어
Adjective	형용사	Alpha	알파벳
Determiner	관형사	Number	숫자
Adverb	부사	KoreanParticle	자음/모음
Conjunction	접속사	URL	웹 주소
Exclamation	감탄사	Hashtag	해시태그(#)
Josa	조사	Email	이메일
PreEomi	선어말어미	ScreenName	아이디 표기(@)
Eomi	어미	Unknown	미등록
Suffix	접미사		

표 5.2 꼬꼬마 품사 태그

태그	품사	태그	품사
NNG	보통명사	EPH	존칭 선어말 어미
NNP	고유명사	EPT	시제 선어말 어미
NNB	일반 의존 명사	EPP	공손 선어말 어미
NNM	단위 의존 명사	EFN	평서형 종결 어미
NR	수사	EFQ	의문형 종결 어미
NP	대명사	EF0	명령형 종결 어미
VV	동사	EFA	청유형 종결 어미
VA	형용사	EFI	감탄형 종결 어미
VXV	보조 동사	EFR	존칭형 종결 어미
VXA	보조 형용사	ECE	대등 연결 어미
VCP	긍정 지정사	ECS	보조적 연결 어미
VCN	부정 지정사	ECD	의존적 연결 어미
MDN	수 관형사	ETN	명사형 전성 어미
MDT	일반 관형사	ETD	관형형 전성 어미

태그	품사	태그	품사
MAG	일반 부사	XPN	체언 접두사
MAC	접속 부사	XPV	용언 접두사
JKS	주격 조사	XSN	명사파생 접미사
JKC	보격 조사	XSV	동사 파생 접미사
JKG	관형격 조사	XSA	형용사 파생 접미사
JK0	목적격 조사	XR	어근
JKM	부사격 조사	SF	마침표, 물음표, 느낌표
JKI	호격 조사	SE	줄임표
JKQ	인용격 조사	SS	따옴표, 괄호표, 줄표
JC	접속 조사	SP	쉼표, 가운뎃점, 콜론, 빗금
JX	보조사	S0	붙임표(물결, 숨김, 빠짐)
OH	한자	SW	기타기호(논리수학기호, 화폐기호)
OL	외국어	IC	감탄사
ON	숫자	UN	명사추정범주

⇒ 시스템의 목적과 환경에 맞는 적절한 형태소 분석기를 사용해야 함

5.3.4. NLTK 라이브러리 실습

NLTK(Natural Language ToolKit) 라이브러리

- 자연어 처리를 위해 개발된 라이브러리
- 형태소 분석, 구문 분석, 개체명 인식, 감성 분석과 같은 기능 제공

- 주로 영어 자연어 처리를 위해 개발됐지만 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공
- NLTK 라이브러리를 활용해 토큰화나 품사 태깅 작업을 하기 위해서는 해당 작업을 수행할 수 있는 패키지나 모델을 다운로드 해야함

실습

- Punkt 모델과 Averaged Perceptron Tagger 모델을 활용해 토큰화 및 품사 태깅 작업 수행
- 두 모델 모두 트리뱅크(TreeBank)라는 대규모의 영어 말뭉치를 기반으로 학습됨
- Punkt 모델: 통계 기반 모델
- Averaged Perceptron Tagger 모델: 퍼셉트론을 기반으로 품사 태깅을 수행

영문 토큰화(Punkt)

- 영문 토큰화는 punkt 모델을 기반으로 단어나 문장을 토큰화
- 단어 토큰나이저(`word_tokenize`): 문장을 입력받아 공백을 기준으로 단어를 분리하고 구두점 등을 처리해 각각의 단어(token)를 추출해 리스트로 반환
- 문장 토큰나이저(`sent_tokenize`): 문장을 입력받아 마침표, 느낌표, 물음표 등의 구두점을 기준으로 문장을 분리해 리스트로 반환
- 영어는 한국어보다 토큰화하기 쉬운 구조를 가지고 있어, 위와 같은 방법으로 간단하게 단어나 문장으로 토큰화를 수행할 수 있음

영문 품사 태깅(Averaged Perceptron Tagger)

- NLTK 라이브러리의 품사 태깅(`pos_tag`) 메서드는 토큰화된 문장에서 품사 태깅을 수행
- 품사 태깅을 수행하기 위해서는 토큰화된 단어들이 들어가야 하며, Averaged Perceptron Tagger 모델이 설치되어 있어야 함
- 35개의 품사를 태깅할 수 있음

태그	품사	태그	품사
CC	접속사	PDT	관사
CD	기수(Cardinal Number)	POS	소유격 조사
DT	관형사	PRP	인칭 대명사
EX	there의 대명사형태	PRP\$	소유격 대명사
FW	외래어	RB	부사

태그	품사	태그	품사
JN	전치사	RBR	비교급 부사
JJ	형용사	RBS	최상급 부사
JJR	비교급 형용사	RP	전치사 또는 조동사
JJS	최상급 형용사	VB	기본형 동사
LS	목록 표시	VBD	과거형 동사
MD	조동사	VBG	현재 부사형 동사
NN	단수형 명사	VBN	과거 분사형 동사
NNS	복수형 명사	VBP	1인칭 단수 현재형 동사
NNP	단수형 고유 명사와 서수(Ordinal Number)	VBZ	3인칭 단수 현재형 동사
NNPS	복수형 고유 명사	WDT	관계사
SYM	기호	WP	주격 관계대명사
TO	To 부정사	WP\$	소유격 관계대명사
UH	감탄사	WRB	관계부사

5.3.5. SpaCy 라이브러리 실습

SpaCy 라이브러리

- 사이썬(Cython) 기반으로 개발된 오픈 소스 라이브러리
- NLTK 라이브러리와 같이 자연어 처리를 위한 기능을 제공
- NLTK 라이브러리와 차이점
 - 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반의 자연어 처리 라이브러리라는 점
 - 더 크고 복잡한 모델
 - 더 많은 리소스를 요구함
- GPU 가속 제공
- 영어, 프랑스어, 한국어, 일본어 등 24개 이상의 언어로 사전 학습된 모델을 제공

실습

- 영어를 대상으로 토큰화 할 예정이므로 영어로 사전 학습된 모델인 `en_core_web_sm` 설치

SpaCy 모델

- 사전 학습된 모델을 기반으로 처리 => 모델 불러오기 함수(`load`)를 통해 모델 설정
- 객체 지향적으로 구현
 - 처리한 결과를 doc 객체에 저장
 - doc 객체는 다시 여러 token 객체로 이뤄져 있음
 - token 객체에 대한 정보를 기반으로 다양한 자연어 처리 작업을 수행
 - nlp 인스턴스에 문장을 입력하면 doc 객체가 반환되며 token 객체의 `tag_` 나 `pos_` 와 같은 속성에 접근해 값을 확인할 수 있음
- token 객체에는 기본 품사 속성(`pos_`), 세분화 품사 속성(`tag_`), 원본 텍스트 데이터(`text`), 토큰 사이의 공백을 포함하는 텍스트 데이터(`text_with_ws`), 벡터(`vector`), 벡터 노름(`vector_norm`) 등의 속성이 포함돼 있음
- SpaCy에서 사용되는 태그

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
ADJ	JJ	형용사	NUM	CD	기수(Cardinal Number)
ADJ	JJR	비교급 형용사	PROPN	NNP	고유 명사
ADJ	JJS	최상급 형용사	PROPN	NNPS	복수형 고유 명사
ADP	IN	전치사	PUNCT	-LRB-	왼쪽 괄호
ADV	RB	부사	PUNCT	-RRB-	오른쪽 괄호
ADV	RBR	비교급 부사	PUNCT	,	쉼표
ADV	RBS	최상급 부사	PUNCT	:	콜론
ADV	WRB	관계 부사	PUNCT	.	마침표, 물음표, 느낌표
AUX	MD	조동사	PUNCT	...	생략 부호
CCONJ	CC	접속사	PUNCT	“	여는 따옴표
DET	DT	관사, 한정사	PUNCT	”	닫는 따옴표
DET	PDT	전치사 한정사	PUNCT	HYPH	하이픈
DET	PRP\$	소유격 대명사	PUNCT	NFP	불필요한 문장 부호
DET	WDT	관계 대명사	SYM	\$	통화
DET	WP\$	소유격 관계 대명사	VERB	VB	동사 기본형
INTJ	UH	감탄사	VERB	VBD	과거형 동사
NOUN	NN	명사	VERB	VBG	동명사, 현재분사
NOUN	NNS	복수형 명사	VERB	VCN	과거분사
PART	POS	소유격 조사	VERB	VBP	동사 현재형
PART	RP	부사적 불변화사	VERB	VBZ	3인칭 단수 동사 현재형

pos_ 속성	tag_ 속성	품사	pos_ 속성	tag_ 속성	품사
PART	T0	To 부정사	X	ADD	이메일
PRON	EX	존재문(there)	X	FW	외래어
PRON	PRP	인칭 대명사	X	LS	목록
PRON	WP	관계 대명사	X	SYM	기호

5.3.6. 형태소 분석과 품사 태깅의 의미

- 형태소 분석과 품사 태깅은 문장에서 의미를 갖는 최소 단위이므로 자연어 처리에서 매우 중요한 전처리 작업임
- 띄어쓰기와 맞춤법이 잘 지켜지지 않는 경우나 신조어, 외래어, 특정 도메인에서 사용되는 축약어 등은 완벽히 대응하기 어려움. 그럼에도 불구하고 형태소 분석과 품사 태깅은 자연어에서 필수적인 작업임. 이를 통해 단어의 의미를 파악하고 문장의 구조를 이해할 수 있음
- 주요 키워드를 추출하거나 문장의 긍정/부정 여부를 판단하는 언어 모델의 성능을 높이는 데도 중요한 역할을 함
 - ⇒ 자연어 처리 분야에서 핵심적인 기술로 자리잡고 있으며, 이를 활용하여 다양한 자연어 처리 응용 프로그램을 개발할 수 있음

5.4. 하위 단어 토큰화(Subword Tokenization)

5.4.1. 하위 단어 토큰화 개념

형태소 토큰화의 한계

- 현대의 일상 언어에서는 맞춤법, 띄어쓰기가 엄격하게 지켜지지 않음
- 신조어, 고유어 등이 빈번하게 생겨남
- 디지털 매체에서 생성되는 텍스트는 오타자가 발생할 확률이 높음

⇒ 형태소 분석기는 모르는 단어를 적절한 단위로 나눈 것에 취약하며, 이는 잠재적으로 어휘 사전의 크기를 크게 만들고 OOV에 대응하기 어렵게 만들

형태소 토큰화의 예시

- 외래어, 띄어쓰기 오류, 오타자 등이 있는 경우

원문: 시보리도 짱짱해고 허리도 어병하지만구 조하효

결과: ['시', '보리', '도', '짱짱해고', '허리', '도', '어', '병하지만구', '조', '하', '효']

- '시보리'라는 외래어어를 인식하지 못하고 '시'와 '보리'로 나뉨
- '어병하다'라는 표현이 '어'와 '병하지만구' 토큰으로 나뉨

- 신조어가 있는 경우

원문: 진짜 멋있네요! 돈썰날만 하네요.

결과: ['진짜', '멋있네요', '!', '돈썰날', '만', '하네요', '!']

- '돈썰날만'은 '돈썰', '날', '만'으로 토큰화하는 게 적절하지만 신조어를 분석하지 못하고 '돈썰날', '만'으로 토큰화
- '돈썰-'이라는 어근에 올 수 있는 모든 어미에 대한 조합이 하나의 토큰으로 인식

⇒ 어휘 사전에 '돈썰내러', '돈썰나', '돈썰내다' 등이 존재하게 돼 어휘 사전의 크기를 크게 만들

⇒ 현대 자연어 처리에서는 신조어의 발생, 오타자, 축약어 등을 고려해야 하기 때문에 분석할 단어의 양이 많아져 어려움을 겪음

⇒ 이를 해결하기 위한 방법 중 하나로 하위 단어 토큰화 사용

하위 단어 토큰화

: 하나의 단어가 빈번하게 사용되는 하위 단어의 조합으로 나누어 토큰화 하는 방법

- 단어의 길이를 줄일 수 있어 처리 속도가 빨라짐
- OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제를 완화할 수 있음
- 하위 단어 토큰화 방법

- 바이트 페어 인코딩
- 워드피스
- 유니그램 모델

5.4.2. 바이트 페어 인코딩(Byte Pair Encoding, BPE)

바이트 페어 인코딩[다이그램 코딩(Digram Coding)] 개념

- 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘
- 초기에는 데이터 압축을 위해 개발됐으나, 자연어 처리 분야에서 하위 단어 토큰화를 위한 방법으로 사용

바이트 페어 인코딩 알고리즘

- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복 → 자주 등장하는 단어는 하나의 토큰으로 토큰화, 덜 등장하는 단어는 여러 토큰의 조합으로 표현
 - 예시) 원문: abracadabra → Step1: AracadAra → Step2: ABcadAS → Step3: CcadC

- 입력 데이터에서 가장 많이 등장한 글자의 빈도수를 측정 → 가장 빈도수가 높은 글자 쌍을 탐색
 1. 원문 'abracadabra'에서 'ab' 글자 쌍이 가장 빈도수가 높음 → 입력 데이터에 없는 새로운 글자 'A'로 치환(치환되는 글자는 어떤 글자를 사용해도 상관 없음)
 2. 동일한 방식으로 다시 탐색 → 'AracadAra'에서 'ra' 글자 쌍이 가장 빈도수가 높음 → 'B'로 치환
 3. 치환된 데이터는 'ABcadAB' → 'AB' 글자 쌍이 가장 빈도가 높으므로 'C'로 치환
 4. 치환된 데이터는 'CcadC' → 더 이상 치환할 수 있는 글자 쌍이 존재하지 않으므로 입력 데이터를 더 이상 압축할 수 없음
- 토큰나이저로써 바이트 페어 인코딩은 자주 등장하는 글자 쌍을 찾아 치환하는 대신 어휘 사전에 추가함

말뭉치에서 바이트 페어 인코딩 적용

- 말뭉치에서 각 단어가 등장한 빈도를 계산해 다음과 같은 빈도 사전과 어휘 사전을 만들었다고 가정

빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
어휘 사전: ['low', 'lower', 'newest', 'widest']

1. 빈도 사전을 바이트 페어 인코딩으로 재구성

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

2. 빈도 사전을 기준으로 가장 자주 등장한 글자 쌍을 찾음

- 빈도 사전에서 'e', 's' 쌍이 'newest'에서 6번, 'widest'에서 3번 등장해 총 9번으로 가장 많이 등장
- 빈도 사전에서 'e'와 's'를 'es'로 병합하고 어휘 사전에 'es'를 추가

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'es', 't', 6), ('w', 'i', 'd', 'es', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es']

3. 동일한 과정을 다시 반복

- 빈도 사전에서 빈도수가 가장 높은 'es', 't'쌍을 'est'로 병합하고 'est'를 어휘 사전에 추가

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est']

4. 이러한 과정을 10번 반복

빈도 사전: ('low', 5), ('low', 'e', 'r', 2), ('n', 'e', 'w', 'est', 6), ('w', 'i', 'd', 'est', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'es', 'est', 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest']

- 'newer', 'wider', 'lowest' 같이 기존 말뭉치에 등장하지 않았던 단어가 입력되더라도 OOV로 처리되지 않고 기존 어휘 사전을 참고해 'new e r', 'wid e r', 'low est'와 같이 토큰화할 수 있음

센텐스피스(Sentencepiece), 코포라(Korpora) 라이브러리 실습

- **센텐스피스 라이브러리**
 - 구글에서 개발한 오픈소스 하위단어 토큰라이저 라이브러리
 - 바이트 페어 인코딩과 유사한 알고리즘을 사용해 입력 데이터를 토큰화하고 단어 사전을 생성
 - 워드피스, 유니코드 기반의 다양한 알고리즘을 지원
 - 사용자가 직접 설정할 수 있는 하이퍼파라미터들을 제공해 세밀한 토큰나이징 기능을 제공
 - SentencePieceTrainer 클래스의 Train 메서드로 토큰라이저 모델을 학습
 - 문자열 입력을 통해 인자들을 전달받음
- **코포라 라이브러리**
 - 국립국어원이나 AI Hub에서 제공하는 말뭉치 데이터를 쉽게 사용할 수 있게 제공하는 오픈소스 라이브러리
 - 파이썬에서 쉽게 사용할 수 있게 API 제공
 - 말뭉치 불러오기(`Korpora.load`)를 통해 말뭉치 데이터를 다운로드
 - corpus
 - 총 433,631개의 청원 데이터가 저장되어 있음
 - train 속성으로 학습 데이터를 가지고 올 수 있음
 - 첫 번째 청원 데이터는 dataset[0]으로 불러올 수 있음
 - petition에 청원 시작일, 동의 수, 제목 등의 정보가 포함되어 있음
- **센텐스피스 토큰라이저 모델**
 - SentencePieceProcessor 클래스를 통해 학습된 모델을 로드
 - 토큰라이저 모델 불러오기(`tokenizer.load`) 메서드를 통해 petition_bpe.model 모델을 로드
 - `encode_as_pieces` 메서드: 문장 토큰화
 - `encode_as_ids` 메서드: 토큰을 정수로 인코딩해 제공
 - 정수 데이터: 어휘 사전의 토큰에 매핑된 ID 값
=> 이 ID 값을 활용해 자연어 처리 모델을 구축
 - 토큰라이저 모델이나 자연어 처리 모델에서 나온 정수는 `decode_ids` 메서드나 `decode_pieces` 메서드를 통해 문자열 데이터로 변환할 수 있음
 - `get_piece_size` 메서드: 센텐스피스 모델에서 생성된 하위 단어의 개수를 반환
 - `id_to_piece` 메서드: 정숫값을 하위 단어로 변환
 - 하위 단어의 개수만큼 반복해 하위 단어 딕셔너리를 구성
 - 토큰 딕셔너리 출력
 - `<unk>`: OOV 발생 시 매핑되는 토큰
 - `<s>`, `</s>`: 문장의 시작 지점과 종료 지점을 표시하는 토큰
 - 센텐스피스 라이브러리는 설정값에 따라 학습 방법이나 어휘 사전 크기가 달라질 수 있음
 - SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토크나이저가 다룰 수 있는 글자의 비율
model_type	토크나이저 알고리즘 - unigram - bpe - char - word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

5.4.3. 워드피스(Wordpiece)

워드피스 토크나이저 개념

- 바이트 페어 인코딩 토크나이저와 유사한 방법으로 학습되지만, 빈도 기반이 아닌 **확률 기반**으로 글자 쌍을 병합
- 학습 과정에서 확률적인 정보 사용
- 모델이 새로운 하위 단어를 생성할 때 이전 하위 단어와 함께 나타날 확률을 계산해 가장 높은 확률을 가진 하위 단어를 선택 → 선택된 하위 단어는 이후에 더 높은 확률로 선택될 가능성이 높음 → 이를 통해 모델이 좀 더 정확한 하위 단어로 분리할 수 있음
 - 각 글자 쌍에 대한 점수 계산

$$score = \frac{f(x, y)}{f(x) \cdot f(y)}$$

- f: 빈도를 나타내는 함수
- x, y: 병합하려는 하위 단어
- f(x, y): x와 y가 조합된 글자 쌍의 빈도
- score: x와 y를 병합하는 것이 적절한지를 판단하기 위한 점수

워드피스의 어휘 사전 구축 방법

- 말뭉치에서 각 단어가 등장한 빈도를 계산해 다음과 같은 빈도 사전과 어휘 사전을 만들었다고 가정

빈도 사전: ('low', 5), ('lower', 2), ('newest', 6), ('widest', 3)
어휘 사전: ['low', 'lower', 'newest', 'widest']

1. 바이트 페어 인코딩과 같이 빈도 사전 내의 모든 단어를 글자 단위로 나눔

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'i', 'd', 'e', 's', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w']

2. score 계산

- 가장 빈번하게 등장한 쌍: 9번 등장한 'e', 's'
 - e: 17번, s: 9번 등장
 - $9 / 17 * 9 = 0.06$
- 'i', 'd' 쌍은 3번 등장

- i: 3번, d: 3번 등장
- $3 / 3*3 = 0.33$

⇒ 'e', 's' 쌍 대신 'i', 'd' 쌍을 병합

빈도 사전: ('l', 'o', 'w', 5), ('l', 'o', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id']

3. 동일한 과정을 반복해 각 글자 쌍에 대한 점수를 계산

- 'l'과 'o' 글자 쌍이 3번 등장
 - l: 7번, o: 7번 등장
 - $3 / 7*7 = 0.14$

⇒ 'l'과 'o' 글자 쌍을 병합

빈도 사전: ('lo', 'w', 5), ('lo', 'w', 'e', 'r', 2), ('n', 'e', 'w', 'e', 's', 't', 6), ('w', 'id', 'e', 's', 't', 3)
어휘 사전: ['d', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w', 'id', 'lo']

- 위 과정을 반복해 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 학습

토크나이저스 라이브러리

- 토크나이저스 라이브러리의 워드피스 API를 이용하면 쉽고 빠르게 토크나이저를 구현하고 학습할 수 있음
- 정규화(Normalization), 사전 토큰화(Pre-tokenization)을 제공
 - 정규화
 - 일관된 형식으로 텍스트를 표준화
 - 모호한 경우를 방지하기 위해 일부 문자를 대체/제거
 - 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등을 제공
 - 사전 토큰화
 - 입력 문장을 토큰화하기 전에 단어와 같은 작은 단위로 나누는 기능을 제공
 - 공백/구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효율적으로 처리하고 모델의 성능을 향상시킬 수 있음

1. 토크나이저(**Tokenizer**)로 워드피스 모델을 불러옴

2. 정규화 방식과 사전 토큰화 방식을 설정

- 정규화 방식
 - normalizers 모듈에 포함된 클래스를 불러와 시퀀스 형식으로 인스턴스를 전달
 - 예제에서는 NFD 유니코드 정규화, 소문자 변환을 사용
- 사전 토큰화 방식
 - pre_tokenizers 모듈에 포함된 클래스를 불러와 적용
 - 예제에서는 공백과 구두점을 기준으로 분리

3. 훈련(**train**) 메서드를 통해 학습에 사용하려는 데이터셋 경로 전달

4. 학습이 완료됐다면 저장(**save**) 메서드로 학습 결과를 저장

- JSON 형태로 저장
- 정규화, 사전 토큰화 등의 메타데이터와 함께 어휘 사전이 저장

- 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
NFKC()	NFKC 유니코드 정규화
Lowercase()	입력 문장의 모든 대문자를 소문자로 변환
Strip()	입력 문장의 양 끝에 있는 공백 제거
StripAccents()	모든 엑센트 기호 제거
Replace()	문자 혹은 정규 표현식을 기반으로 입력 문장 변환
BertNormalizer()	BERT 모델에 사용된 정규화 적용
Sequence()	여러 개의 정규화 모듈을 병합해 순서대로 수행

- 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
CharDelimiterSplit()	문자 기준으로 문자열을 나눈다.
Digits()	모든 형태의 숫자 표현을 기준으로 문자열을 나눈다.
Punctuation()	구두점을 기준으로 입력 문자열을 나눈다.
Metaspace(replacement="_ ")	Whitespace 기준으로 사전 토큰화를 진행하고, replacement로 Whitespace를 치환한다. 기본값: _(U+2581)
Whitespace()	단어 경계(공백과 구두점)를 기준으로 사전 토큰화를 진행한다. \w+!([^\w\s])+ 정규 표현식을 적용한다.
WhitespaceSplit()	공백 문자를 기준으로 입력 문자열을 나눈다.
Split(pattern, behavior)	정규표현식(pattern)과 동작(behavior)을 기준으로 문자열을 나눈다. - behavior - removed: 삭제 - isolated: 개별 토큰 처리 - merged_with_previous: 이전 문자열과 결합해 토큰 처리 - merged_with_next: 다음 문자열과 결합해 토큰 처리 - contiguous: 다음 문자열이 토큰화되지 않아도 결합해 토큰 처리

워드피스 토큰화

- 모델 결과가 저장된 JSON 파일을 불러와 Tokenizer 객체를 생성
- `WordPieceDecoder()` 를 사용해 Tokenizer의 디코더를 워드피스 디코더로 설정
- 인코딩(`encode`) 메서드로 문장을 토큰화
- 인코딩 배치(`encode_batch`) 메서드로 여러 문장을 한 번에 토큰화
- 인코딩 메서드, 인코딩 배치 메서드로 모두 워드피스 토큰라이저를 통해 문장을 토큰화, 각 토큰의 색인 번호를 반환
- 토큰(`tokens`) 속성을 이용해 토큰화된 데이터 값을 확인
- 토큰 정수(`ids`) 속성으로 인코딩된 문장의 ID 값을 출력
- 디코딩(`decode`) 메서드를 통해 정수 인코딩된 결과를 다시 문장으로 디코딩

5.5. 최근 연구 동향

- 더 큰 말뭉치를 사용해 모델을 학습
- OOV의 위험을 줄이기 위해 하위 단어 토큰화를 활용
- 소개된 알고리즘 외에도
 - 바이트 수준 바이트 페어 인코딩(BBPE): 유니코드 단어가 아닌 바이트 단위에서 토큰화
 - 유니그램(Unigram): 크기가 큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습

Week10_Ch06. 자연어 처리-임베딩(~ TF-IDF)

6.1. 임베딩 개요

6.1.1. 텍스트 벡터화(Text Vectorization)

- 컴퓨터는 텍스트 자체를 이해할 수 없으므로 텍스트를 숫자로 변환하는 텍스트 벡터화 과정이 필요
- 기초적인 텍스트 벡터화
 - 원-핫 인코딩(One-Hot Encoding)
 - 빈도 벡터화(Count Vectorization)

원-핫 인코딩(One-Hot Encoding)

- 문서에 등장하는 각 단어를 고유한 인덱스 값으로 매핑 → 해당 인덱스 위치를 1로 표시하고 나머지 위치는 모두 0으로 표시
- 예시
 - 'I like apples', 'I like bananas' 문장을 띄어쓰기 기준으로 토큰화하고 고유 인덱스 값으로 매핑

색인	0	1	2	3
토큰	I	like	apples	bananas

- 이를 바탕으로 각 문장을 원-핫 인코딩
 - I like apples: [1, 1, 1, 0]
 - I like bananas: [1, 1, 0, 1]
- 같은 단어가 여러 번 등장하더라도 1과 0으로 표현

빈도 벡터화(Count Vectorization)

- 0, 1이 아닌 단어의 빈도로 표현

텍스트 벡터화의 장점

- 단어나 문장을 벡터 형태로 변환하기 쉽고 간단

텍스트 벡터화의 문제점

- 희소성(Sparsity)이 큼
- 말뭉치 내에 존재하는 토큰의 개수만큼의 벡터 차원을 가져야 함
- 하지만 입력 문장 내에 존재하는 토큰의 수는 그에 비해 현저히 적음

⇒ 컴퓨팅 비용의 증가와 차원의 저주와 같은 문제를 겪을 수 있음

- 텍스트의 벡터가 입력 텍스트의 의미를 내포하고 있지 않음 ⇒ 두 문장이 의미적으로 유사하다고 해도 벡터가 유사하게 나타나지 않을 수 있음
 - 예시) 'i scream for ice cream', 'ice cream for i scream'에 원-핫 인코딩이나 빈도 벡터화 적용
⇒ [1, 1, 1, 1, 1], [1, 1, 1, 1, 1]로 반환

⇒ 이러한 문제를 해결하기 위해 단어의 의미를 학습해 표현하는 워드 임베딩 기법을 사용

6.1.2. 워드 임베딩(Word Embedding)

- 단어를 고정된 길이의 실수 벡터로 표현하는 방법
- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론

6.1.3. 동적 임베딩(Dynamic Embedding)

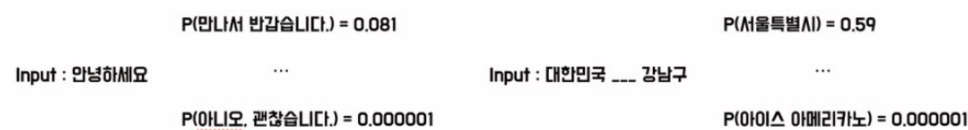
- 워드 임베딩은 고정된 임베딩을 학습하기 때문에 문맥 정보를 다루기 어려움 ⇒ 인공 신경망을 활용해 **동적 임베딩(Dynamic Embedding)** 기법을 사용

6.2. 언어 모델(Language Model)

6.2.1. 언어 모델 개념

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 주어진 문장을 바탕으로 문맥을 이해 → 문장 구성에 대한 예측을 수행
- 자동 번역, 음성 인식, 텍스트 요약 등 다양한 자연어 처리 분야에서 활용

간략한 언어 모델 구조



- ‘안녕하세요’라는 문장 뒤에 어떤 문장이 어떤 확률로 등장할지 계산하는 것은 어려움
⇒ 비지도 학습 문제로 모델 스스로 문장 등장 확률을 계산해야 함
- ‘안녕하세요’ 다음에 올 수 있는 문장은 무한에 가까움
⇒ 완성된 문장 단위로 확률을 계산하는 것은 불가능
- 주어진 문장 뒤에 나올 수 있는 문장은 매우 다양함
⇒ 완성된 문장 단위로 확률을 계산하는 것은 어려움
⇒ 이를 해결하기 위해 문장 전체를 예측하는 방법 대신에 하나의 토큰 단위로 예측하는 방법인 **자기회귀 언어 모델**이 고안

6.2.2. 자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측

언어 모델의 조건부 확률

: 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것

- 이전에 등장한 모든 토큰의 정보를 고려
- 문장의 문맥 정보를 파악하여 다음 단어를 생성
- 다음 단어는 다시 이전 단어를 기반으로 예측이 이루어짐(이 과정을 반복)
- 수식

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

- 언어 모델에서 조건부 확률을 계산하기 위해 이전에 등장한 단어 시퀀스($w_1, w_2, \dots, w_{(t-1)}$)를 기반으로 다음 단어(w_t)의 확률을 계산

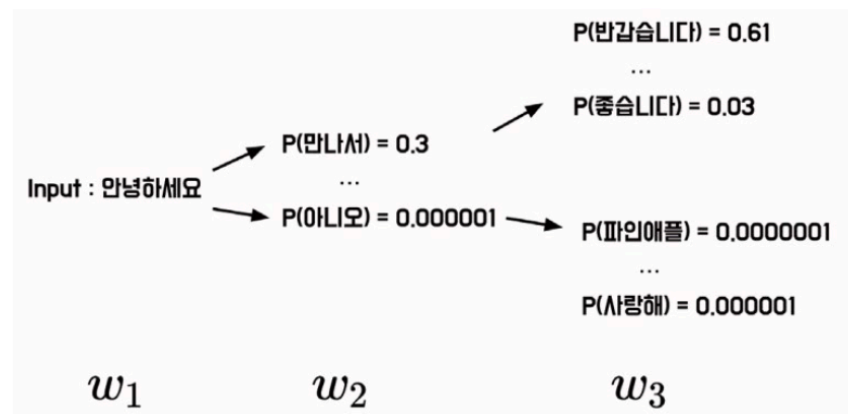
조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

: 하나의 사건이 일어날 확률을 다른 사건들의 조건부 확률을 이용해 계산하는 방식

- 언어 모델에서 조건부 확률은 연쇄법칙을 이용해 계산
- 이전 단어들의 시퀀스가 주어졌을 때, 다음에 등장하는 단어의 확률을 이전 단어들의 조건부 확률을 이용해 계산
- 수식

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \dots P(w_t | w_1, w_2, \dots, w_{t-1})$$

- 문장 전체의 확률 = 첫 번째 단어의 확률 $P(w_1)$ * 각 단어가 이전 단어들의 조건부 확률에 따라 발생할 확률
- 이를 통해 언어 모델은 문장 전체의 확률을 계산하고 그것을 이용해 다음 단어를 예측
- 예시



- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
- 1. 입력 토큰 w_1 을 바탕으로 모델은 다음 토큰 w_2 가 등장할 확률 $P(w_2 | w_1)$ 을 예측
- 2. 모델의 출력값 w_2 가 다시 입력값이 되어, 모델은 확률 $P(w_3 | w_1, w_2)$ 를 예측

자기회귀 언어 모델의 특징

- 모델의 출력값이 모델의 입력값으로 사용됨
 - ⇒ 자기회귀라는 이름이 붙음
- 시점별로 다음에 올 토큰을 예측
 - ⇒ 토큰 분류 문제로 정의할 수 있음
- 이전에 등장한 모든 토큰의 정보를 활용해 입력된 문장의 문맥 정보를 파악하고 다음 토큰을 예측
 - ⇒ 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측

6.2.3. 통계적 언어 모델(Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
- 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악 → 다음에 등장할 단어의 확률을 예측
- 일반적으로 통계적 언어 모델은 마르코프 체인을 이용해 구현

마르코프 체인(Markov Chain)

- 빈도 기반의 조건부 확률 모델 중 하나
- 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측
- 주어진 데이터에서 각 변수가 발생한 빈도수를 기반으로 확률을 계산
 - 예시
 - 말뭉치에 '안녕하세요'라는 문장이 1,000번 등장하고 이어서 '안녕하세요 만나서'가 700번, '안녕하세요 반갑습니다'가 100번 등장했다고 가정
 - 빈도 기반의 조건부 확률 수식

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

통계적 언어 모델 단점

- 빈도 기반의 조건부 확률 방식은 순서와 빈도에만 기초해 문장의 확률을 예측
- 문맥을 제대로 파악하지 못하면 불완전하거나 부적절한 결과를 생성할 수 있음
- 한 번도 등장한 적이 없는 단어나 문장에 대해서는 정확한 확률을 예측하기가 어려움

⇒ **데이터 희소성(Data Sparsity)**: 관측한 적이 없는 데이터를 예측하지 못하는 문제

통계적 언어 모델 장점

- 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포를 생성
 - ⇒ 새로운 문장을 생성할 수 있으며, 다양한 종류의 텍스트 데이터를 학습할 수 있음
- 대규모 자연어 데이터를 처리하는 데 효과적
- 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델을 구현할 수 있게 됨

최근 연구되는 자연어 처리 기법

- 언어 모델을 활용해 가중치를 사전 학습
- GPT(Generative Pre-trained Transformer)

: 트랜스포머 모델에 기반한 언어 모델

- 문장 생성 기법

Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
Input: GPT는 OpenAI에서 개발한
Prediction-1: 인과적
Prediction-2: 언어 모델입니다.

- BERT(Bidirectional Encoder Representations from Transformers)

- 문장 생성 기법

Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
Prediction: Google, 마스킹된

- GPT, BERT 모델
 - 자연어 처리 분야에서 혁신적인 성과를 거둔 언어 모델 중 하나
 - 이렇게 신경망을 통해 확률 P를 계산할 수도 있지만, 통계적인 방법을 통해 확률을 계산할 수 있음

6.3. N-gram

6.3.1. N-gram 개념

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정
- 입력 텍스트를 하나의 토큰 단위로 분석하지 않고 N개의 토큰을 묶어서 분석
 - 연속된 N개의 단어를 하나의 단위로 취급하여 추론하는 모델
 - N = 1일 때 유니그램(Unigram)
 - N = 2일 때 바이그램(Bigram)
 - N = 3일 때 트라이그램(Trigram)

Unigram : <u>안녕하세요</u> 만나서 <u>진심으로</u> 반가워요	안녕하세요. 만나서. <u>진심으로</u> . 반가워요
Bigram : <u>안녕하세요 만나서</u> <u>진심으로</u> 반가워요	안녕하세요 만나서. 만나서 <u>진심으로</u> . <u>진심으로</u> 반가워요
Trigram : <u>안녕하세요 만나서</u> <u>진심으로</u> 반가워요	안녕하세요 만나서 <u>진심으로</u> . 만나서 <u>진심으로</u> 반가워요

- $N \geq 4$ 일 때 N-gram

N-gram 언어 모델 수식

- 모든 토큰을 사용하지 않고 (N-1)개의 토큰만을 고려해 확률을 계산

- 각 N-gram 언어 모델에서 t번째 토큰의 조건부 확률을 계산하는 수식

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- w_t : 예측하려는 단어
- $w_{(t-1)} \sim w_{(t-N+1)}$: 예측에 사용되는 이전 단어들
- 이전 단어들의 개수를 결정하는 N의 값을 조정하여 N-gram 모델의 성능을 조절할 수 있음

6.3.2. N-gram 실습

- 간단한 파이썬 코드 또는 NLTK 라이브러리로 구현
 - 간단한 파이썬 코드

```
def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)

print(unigram)
print(bigram)
print(trigram)
```

- NLTK 라이브러리

```
import nltk

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams(sentence.split(), 2)
trigram = nltk.ngrams(sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

- 파이썬 코드로 구현한 N-gram과 NLTK 라이브러리에서 지원하는 N-gram 함수는 동일한 출력값을 반환

6.3.3. N-gram 장점, 활용

- 작은 규모의 데이터셋에서 연속된 문자열 패턴을 분석하는 데 큰 효과를 보임
- 관용적 분석에 활용
 - 예) '입이 무겁다'라는 표현은 '비밀을 잘 지킨다'하는 뜻을 가지고 있음.
 - 자주 등장하는 연속된 단어나 구를 추출하고, 이를 분석함으로써 관용적 표현을 파악할 수 있음
- 시퀀스에서 연속된 n개의 단어를 추출하므로 단어의 순서가 중요한 자연어 처리 작업 및 문자열 패턴 분석에 활용

6.4. TF-IDF(Term Frequency-Inverse Document Frequency)

6.4.1. TF-IDF 개요

TF-IDF(Term Frequency-Inverse Document Frequency)

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법
- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적인 가중치

⇒ BoW(Bag-of-Words)에 가중치를 부여하는 방법

BoW(Bag-of-Words)

- 문서나 문장을 단어의 집합으로 표현하는 방법
- 문서나 문장에 등장하는 단어의 중복을 허용해 빈도를 기록
- 원-핫 인코딩은 단어의 등장 여부를 판별해 0/1로 표현하지만 BoW는 등장 빈도를 고려해 표현
 - 예시) ['That movie is famous movie', 'I like that actor', 'I don't like that actor'] 말뭉치를 BoW로 벡터화

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

- BoW를 이용해 벡터화 ⇒ 모든 단어는 동일한 가중치를 가짐
 - 영화 리뷰의 긍/부정 분류 모델을 만든다면 높은 성능을 얻기 어려움
 - 'I like this movie'와 'I don't like this move'는 'don't'라는 단어가 문장 분류에 결정적인 역할을 함
 - 하지만 'don't'는 자주 등장하지 않으므로 토큰화나 분류 모델 진행 시 데이터가 무시될 수 있음

6.4.2. 단어 빈도(Term Frequency, TF)

- 문서 내에서 특정 단어의 빈도수를 나타내는 값
 - 예시) 3개의 문서에서 'movie'라는 단어가 4번 등장 ⇒ 해당 단어의 TF 값은 4

수식

$$TF(t, d) = count(t, d)$$

- 문서 내에서 단어가 등장한 빈도수를 계산
- 해당 단어의 상대적인 중요도를 측정하는 데 사용
- TF 값이 높을수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 생각할 수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주할 수 있음
- 단순히 단어의 등장 빈도수를 계산 ⇒ 문서의 길이가 길어질수록 해당 단어의 TF 값도 높아질 수 있음

6.4.3. 문서 빈도(Document Frequency, DF)

- 한 단어가 얼마나 많은 문서에 나타나는지
- 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수를 계산
 - 예시) 3개의 문서에서 'movie'라는 단어가 4번 등장 ⇒ 해당 단어의 DF 값은 3

수식

$$DF(t, D) = count(t \in d : d \in D)$$

- 단어가 몇 개의 문서에서 등장하는지 계산
- DF 값이 높으면 특정 단어가 많은 문서에서 등장
 - ⇒ 그 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있음
- DF 값이 낮으면 특정 단어가 적은 수의 문서에만 등장

⇒ 특정한 문맥에서만 사용되는 단어일 가능성이 있으며, 중요도가 높을 수 있음

6.4.4. 역문서 빈도(Invers Document Frequency, IDF)

- 전체 문서 수를 문서 빈도로 나눈 다음에 로그를 취한 값
- 문서 내에서 특정 단어가 얼마나 중요한지 표현
- 문서 빈도가 높을수록 일반적이고 상대적으로 중요하지 않다는 의미
 - ⇒ 문서 빈도에 역수를 취하면 단어의 빈도수가 적을수록 IDF 값이 커지게 보정하는 역할을 함
 - ⇒ 문서에서 특정 단어의 등장 횟수가 적으면 IDF는 상대적으로 커짐

수식

$$IDF(t, D) = \log\left(\frac{count(D)}{1 + DF(t, D)}\right)$$

- 분모 = 1 + DF 값
 - 특정 단어가 한 번도 등장하지 않는다면 분모가 0이 되므로 작은 값을 더해 분모가 0이 되는 걸 방지
- 로그
 - 전체 문서 수를 문서 빈도로 나눈 값을 사용한다면 너무 큰 값이 나올 수 있음
 - 10,000개의 문서에서 특정한 단어가 1번만 등장하면 IDF 값은 5,000이 됨
 - 이러한 문제점을 방지하고자 로그를 취해 정교한 가중치를 얻음

6.4.5. TF-IDF

수식

$$TF - IDF(t, d, D) = TF(t, d) * IDF(t, d)$$

- 문서 빈도 * 역문서 빈도
- 문서 내에 단어가 자주 등장하지만, 전체 문서 내에서 해당 단어가 적게 등장
 - ⇒ TF-IDF 값 커짐
- 전체 문서에서 자주 등장할 확률이 높은 관사, 관용어 등
 - ⇒ TF-IDF 값(가중치) 작아짐

사이킷런을 이용한 계산(실습)

- 사이킷런 라이브러리
 - 파이썬의 대표적인 머신러닝 라이브러리
 - 머신러닝에 필요한 전처리, 모델 파이프라인, 모델 검증 등에 필요한 전반적인 모듈을 제공
 - 말뭉치를 쉽게 TF-IDF 형태로 변환할 수 있게 TF-IDF 클래스 제공
- TF-IDF 클래스의 주요 매개 변수
 - 입력값(`input`)
 - 입력될 데이터의 형태
 - 기본값: content(문자열/바이트 형태)
 - 파일 객체를 사용한다면 file로 입력
 - 파일 경로를 사용하는 경우 filename으로 입력
 - 인코딩(`encoding`)
 - 바이트 혹은 파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
 - 소문자 변환(`lowercase`)
 - 입력받은 데이터를 소문자로 변환할지 여부

- True ⇒ 모든 입력 텍스트를 소문자로 변환
- 불용어(`stop_words`)
 - 분석에 도움이 되지 않는 의미 없는 단어들
 - 입력받은 단어들은 단어 사전에 추가되지 않음
- N-gram 범위(`ngram_range`)
 - 사용할 N-gram의 범위
 - (최솟값, 최댓값) 형태로 입력
 - (1, 1): 유니그램
 - (1, 2): 유니그램&바이그램
- 최댓값 문서 빈도(`max_df`)
 - 전체 문서 중 일정 횟수 이상 등장하는 단어는 불용어로 처리
 - 정수 입력 => 해당 등장 횟수를 초과해 등장하는 단어를 불용어 처리
 - 1 이하의 실수 입력 => 해당 비율을 초과해 등장한 단어를 불용어 처리
- 최솟값 문서 빈도(`min_df`)
 - 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리
 - 최댓값 문서 빈도와 동일한 패턴으로 입력
- 단어사전(`vocabulary`)
 - 미리 구축한 단어사전이 있다면 해당 단어 사전을 사용
 - 입력하지 않는다면 TF-IDF 학습 시 자동으로 구축
- IDF 분모처리(`smooth_idf`)
 - IDF 계산 시 분모에 1을 더함
- TF-IDF 클래스
 - `fit` 메서드
 - fit 메서드를 통해 학습해야 데이터의 변환/추론이 가능
 - TF-IDF 클래스를 통해 `tfidf_vectorizer` 객체를 불러오고, fit 메서드를 통해 준비된 말뭉치를 학습
 - `transform` 메서드
 - 객체의 학습이 완료되면 transform 메서드를 이용해 데이터 변환을 수행
 - `fit_transform` 메서드를 통해 학습과 변환을 동시에 수행할 수도 있음
 - `toarray` 메서드
 - TF-IDF를 넘파이 배열로 변환
 - (문서 수)*(단어 수)의 형태를 가짐
 - 각 행은 하나의 문서에 해당. 각 열은 단어를 의미
 - `vocabulary_` 속성
 - TF-IDF에 사용된 단어 사전
 - 딕셔너리의 키: 고유한 단어
 - 딕셔너리의 값: 해당 단어에 대한 색인 값
 - 점수가 가장 높은 값을 세 개만 추려 색인으로 정리
 - `[[2, 3, 5], [0, 4, 6], [0, 1, 4]]`
 - 사전으로 매핑
 - `[[famous, is, movie], [actor, like, that], [actor, don, like]]`
 - 이를 통해 문서마다 중요한 단어만 추출할 수 있으며, 벡터값을 활용해 문서 내 핵심 단어를 추출할 수 있음

- 하지만 빈도 기반 벡터화는 문장의 순서나 문맥을 고려하지 않음 => 문장 생성과 같이 순서가 중요한 작업에는 부적합
- 벡터가 해당 문서 내의 중요도를 의미할 뿐, 벡터가 단어의 의미를 담고 있지는 않음
 - 예시) ['나는 바나나를 먹었다.', '그녀가 과일을 섭취한다', '고양이가 야옹 운다']를 벡터화 하면 '나는 바나나를 먹었다'보다 '그녀가 과일을 섭취한다'가 '고양이는 야옹 운다'보다 더 가까운 의미를 가지고 있지만, 세 문장 간의 유사도는 동일